Machine Coding: Click to Copy (Clipboard)

Problem: Build a reusable "click to copy" feature: a value is shown next to a Copy button; clicking it copies the value to the clipboard and gives clear feedback. Real-world uses: coupon codes, meeting invite links, API keys, code snippets, OTPs.

Tech stack: React 19 (Vite, JavaScript, plain CSS, `react-icons`) | **Time limit:** 60 minutes

---

PART A: Thought process (works for any machine coding task)

1. **Clarify** what must work and what is out of scope. Write the assumptions down.
2. **List the interactions** (user action → expected result). This is your checklist.
3. **Sketch the components** and who owns which state.
4. **Design the state** first, the UI second. Store the minimum, derive the rest.
5. **Name the hard parts** before coding, so you budget time for them.
6. **Build the simplest working version**, then add one working layer at a time.
7. **Walk the states and edge cases** (initial, success, error, empty, rapid input).
8. **Wrap up**: say what you handled, what you traded off, what you would add next.

Time budget for 60 minutes

Minutes	Activity
0 – 7	Requirements, clarifying questions, assumptions
7 – 15	Interactions table, components, state design
15 – 40	Build: basic copy → hook → feedback timer → fallback → reuse
40 – 50	Edge cases, accessibility, styling
50 – 60	Test every row of the interactions table, wrap-up

> **Think out loud.** The interviewer grades your reasoning, not only the result. Say *why* before each decision ("I clear the timer after the await because a second click can start one while we wait").

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a value (text, link, secret, multi-line code) with a **Copy** button.

- Clicking **Copy** puts the exact value on the system clipboard.
- The button gives feedback: **Copied** (success) or **Failed** (error).
- Feedback returns to **Copy** after ~2 seconds.
- Secrets (API keys) are **masked on screen** but the **real value is copied**.
- Many copyable fields can be on one page, each with its own feedback.

Non-functional requirements

Concern	What it means here
Performance	No re-render of other fields when one is copied (state is local to each button).
Scalability	Adding a new copyable item = one data object, no new code.
Error handling	Clipboard API can be missing (http, old browser) or denied (permission, iframe). Fall back, then show an error, never crash.
Accessibility	Real `<button>`, keyboard usable, visible focus ring, screen reader hears "copied" via an `aria-live` region.
Reusability	Copy logic lives in a hook (`useCopyToClipboard`), usable by any component.
Extensibility	Reset delay, labels and icons are easy to change; hook can be reused for "copy row", "share link", etc.

Clarifying questions to ask

1. Which browsers must work? Do we need to support pages served over **http** (no secure context)?
2. Should only **one** field show "Copied" at a time, or can each field show its own feedback?
3. Do we copy **plain text only**, or rich content (HTML, images)?
4. How long should the "Copied" feedback stay? Should we show a toast instead of changing the button?
5. For secrets: mask on screen? Do we need a "reveal" toggle?
6. Should we track copies (analytics) or auto-clear the clipboard for sensitive data?

Assumptions we build with

- Plain text only. Modern browsers, plus a fallback for http / old browsers.
- Each field has **independent** feedback (two fields can show "Copied" at once).
- Feedback stays **2 seconds**, then resets. A new click restarts the 2 seconds.
- Secrets are masked with a fixed number of bullets + last 4 chars; no reveal toggle.
- Data is local (no API); items come from a static array.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Every field shows its value and a **Copy** button
Click **Copy** on coupon	Clipboard = `DIWALI50`; button turns green "Copied"; screen reader hears "Coupon code copied to clipboard"

Action / input	Expected behavior / output
Wait 2 s	Button returns to **Copy**
Click **Copy** again within 2 s	Copies again; the 2 s timer restarts (no early reset)
Click **Copy** on API key	Screen shows `••••••••aZ1c`, clipboard gets the **full** key
Click **Copy** on code snippet	Multi-line text copied with line breaks intact
Press `Tab` to the button, press `Enter` / `Space`	Same as click (native button)
Page on http / Clipboard API missing	Fallback copy via hidden textarea still works
Clipboard permission denied **and** fallback fails	Button turns red "Failed"; screen reader hears "Could not copy …"
Copy field A, then field B	Each shows its own feedback; A is not affected by B
Component unmounts while "Copied"	Timer cleared; no state update after unmount

This table is the build checklist and the test list.

---

3. Design the architecture

Module	Responsibility
`utils/clipboard.js`	Pure-ish logic: `copyText(text)` (modern API + fallback), `maskSecret(value)`
`hooks/useCopyToClipboard.js`	Owns status `idle / copied / error` and the reset timer
`components/CopyButton.jsx`	Button + icon + label + `aria-live` announcement; uses the hook
`components/CopyField.jsx`	Label + displayed value (masked/code/plain) + `CopyButton`
`data/items.js`	The list of copyable items
`App.jsx`	Renders the list

```
App
 └─ CopyField (× N, from COPY_ITEMS)
      ├─ value display (<code> / <pre> / masked)
      └─ CopyButton
           └─ useCopyToClipboard()  ──▸ copyText()  ──▸ navigator.clipboard
                                                    └▸ execCommand fallback
```

Start simple: the first version is a single button with `navigator.clipboard.writeText` inline. Split into hook + utils only once the second and third use cases (masking, fallback, timer) appear.

---

4. Design the state / data model

State or field	Purpose
`status` (`"idle" | "copied" | "error"`)	What the button shows; one enum instead of two booleans
`timerRef` (ref, not state)	Id of the pending reset timer, so we can clear it; changing it must not re-render
`item.value`	The real value that is copied
`item.kind` (`"text" | "secret" | "code"`)	How the value is **displayed**

```
// inside useCopyToClipboard
const [status, setStatus] = useState("idle"); // idle | copied | error
const timerRef = useRef(null);
```

Senior notes

- **One enum, not `copied` + `error` booleans.** Two booleans allow the impossible state `copied && error`.
- **Derived, not stored:** the button label, icon, CSS class and the screen-reader message are all derived from `status`. The masked text is derived from `value` + `kind` at render.
- **Ownership:** each `CopyButton` owns its own status (colocated). The parent never knows which field is copied, so copying one field re-renders only that button.
- **Timer id in a ref:** it is bookkeeping, not UI.

---

5. Design the data structures and contracts

Example data

```
export const COPY_ITEMS = [
  { id: "coupon",  label: "Coupon code",         value: "DIWALI50", kind: "text" },
  { id: "invite",  label: "Meeting invite link", value: "https://meet.google.com/abc-defg-hij", kind:
"text" },
  { id: "api-key", label: "API key",             value: "sk_live_51HxQ2eK9vT7mN3pR8wL4aZ1c", kind:
"secret" },
  { id: "otp",     label: "One-time password",   value: "482913", kind: "text" },
  { id: "snippet", label: "Setup commands",      value: "npm install react-icons\nnpm run dev", kind:
"code" },
];
```

- **Stable `id`** → React `key`. Never the index: if the list is filtered or reordered, an index key would move one button's "Copied" state onto another field.
- **`label`** is reused for the aria-label and the announcement ("API key copied to clipboard").

- `kind` only changes display. The copied value is always `value`.

Contracts (no real API, so the contracts are function signatures)

Function / component	Input	Output
`copyText(text)`	non-empty `string`	`Promise<void>`; resolves on success, **rejects** on empty input or when every method fails
`maskSecret(value, visible = 4)`	`string`	`"••••••••" + last 4` (fixed 8 bullets, does not leak length)
`useCopyToClipboard(resetAfter = 2000)`	ms	`{ status, copy(text) }`
`<CopyButton text label />`	value to copy, human label	button + live region
`<CopyField item />`	one item object	full row

---

6. Trace the data flow (click Copy on the API key)

```
User clicks "Copy" on API key
        │
        ▼
CopyButton onClick → copy("sk_live_51Hx…aZ1c")          (real value, not the masked one)
        │
        ▼
copyText(): secure context + navigator.clipboard?
        ├── yes → await navigator.clipboard.writeText(text)
        │         └── rejects (permission) → legacy fallback
        └── no  → legacy fallback: hidden textarea → select() → execCommand("copy") → remove → restore
focus
        │
        ▼
hook: next = "copied" (or "error" if copyText threw)
      clearTimeout(old timer) → setStatus(next) → start 2 s timer
        │
        ▼
Render: button green "Copied" ✓, live region says "API key copied to clipboard"
        │
        ▼ (2 s later)
timer → setStatus("idle") → button back to "Copy", live region empty
```

---

7. Important technical challenges

Challenge 1: The Clipboard API is not always available

Problem. `navigator.clipboard` exists only in a **secure context** (https or localhost). On `http://192.168.1.5:5173` (testing on a phone) it is `undefined` and `writeText` crashes. Even on https, `writeText` can **reject**: permission denied, a cross-origin iframe without `clipboard-write`, or the document not focused.

Solution. Try the modern API; on absence or rejection, fall back to the legacy `execCommand("copy")` trick with a hidden textarea. If that also fails, throw so the UI can show "Failed".

```
export async function copyText(text) {
  if (typeof text !== "string" || text.length === 0) throw new Error("Nothing to copy");
  if (navigator.clipboard && window.isSecureContext) {
    try { await navigator.clipboard.writeText(text); return; } catch { /* fall through */ }
  }
  legacyCopy(text);
}
```

Challenge 2: The fallback has side effects (scroll jump, keyboard, lost focus)

Problem. The hidden-textarea trick, written naively: appending a textarea at the bottom of the page scrolls there; on mobile, focusing it opens the keyboard; `select()` **steals focus** from the button, so a keyboard user's focus is lost after copying.

Solution. `position: fixed` (no scroll), `readonly` (no keyboard), `opacity: 0`, remove it in `finally` (even on error), and restore focus to the element that had it.

```
const previousFocus = document.activeElement;
const textarea = document.createElement("textarea");
textarea.value = text;
textarea.setAttribute("readonly", "");
textarea.style.position = "fixed";
textarea.style.opacity = "0";
document.body.appendChild(textarea);
textarea.select();
try {
  if (!document.execCommand("copy")) throw new Error("Copy command was rejected");
} finally {
  document.body.removeChild(textarea);
  previousFocus?.focus();
}
```

Challenge 3: The "Copied" timer: rapid clicks, async race, unmount

Problem.

- Click at 0 s, click again at 1.5 s. If the first timer is not cleared, it resets the button at 2 s, only 0.5 s after the second click.
- `copyText` is **async**. Two quick clicks both `await`; if we clear the timer *before* the await, both clicks then start timers and one id is overwritten and never cleared.
- The component unmounts (route change) while a timer is pending → a state update after unmount, i.e. a leak.

Solution. Keep the timer id in a ref. Clear it **after** the await, right before starting a new one. Clear it on unmount.

```
const copy = useCallback(async (text) => {
  let next = "copied";
  try { await copyText(text); } catch { next = "error"; }
  clearTimeout(timerRef.current);          // after the await, see above
  setStatus(next);
  timerRef.current = setTimeout(() => setStatus("idle"), resetAfter);
}, [resetAfter]);

useEffect(() => () => clearTimeout(timerRef.current), []);
```

Challenge 4: Display ≠ copied value, and feedback a screen reader can hear

Problem. The API key must be masked on screen, but the user needs the real key on the clipboard. A green check is invisible to a screen reader user, who gets no confirmation that anything happened.

Solution. `CopyField` passes the **real** `value` to `CopyButton` and only masks the display. `CopyButton` renders a visually hidden `role="status" aria-live="polite"` region whose text is derived from `status`.

```
const shown = kind === "secret" ? maskSecret(value) : value;
// ...
<CopyButton text={value} label={label} />

// in CopyButton
<span className="sr-only" role="status" aria-live="polite">{announcement}</span>
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Hard-code one value + button calling `navigator.clipboard.writeText`	Copy works on localhost
2	Add `copied` feedback with `setTimeout`	Button shows "Copied" for 2 s
3	Move logic into `useCopyToClipboard` with `status` enum + `timerRef`	Same behavior, reusable
4	Clear the timer on re-click and on unmount	No early reset, no leak

#	Step	Working result after the step
5	Extract `copyText` to `utils/clipboard.js` with input validation	Logic testable without React
6	Add the `execCommand` fallback + focus restore	Works on http / denied permission
7	Add the `error` status and red "Failed" state	Failures are visible
8	Create `COPY_ITEMS` + `CopyField`, map with stable ids	Five independent fields
9	Add `kind`: masked secret, `<pre>` for code	API key masked, snippet multi-line
10	Accessibility: aria-label, live region, focus ring	Screen reader + keyboard OK
11	Styling + icons	Polished UI
12	Walk the interactions table + edge cases	Verified

9. Cover all states and edge cases

State / edge case	What the user sees	How it is handled
Initial	Value + "Copy" button	`status = "idle"`
Processing (await)	Still "Copy" (copy takes ms)	No spinner needed; status changes after await
Success	Green "Copied" ✓ + announcement	`status = "copied"`
Error	Red "Failed" ✗ + announcement	`status = "error"` when every method fails
Reset	Back to "Copy" after 2 s	Timer sets `idle`
Rapid double click	Stays "Copied" a full 2 s after the last click	Clear old timer after the await
Empty value	"Failed"	`copyText` rejects `""`
http / no Clipboard API	Copy still works	Legacy fallback
Permission denied	Copy still works or "Failed"	Rejection → fallback → error
Secret value	`••••••••aZ1c` on screen, full key copied	Mask display only
Multi-line snippet	Line breaks kept	`<pre>`, raw string copied
Long link	Scrolls inside its box, page doesn't overflow	`overflow-x: auto; min-width: 0`
Keyboard user	Tab + Enter/Space copies, focus stays on button	Native `<button>`, focus restore in fallback
Unmount mid-feedback	Nothing	Cleanup clears timer
Two fields copied	Both show "Copied" independently	State per button

10. Final structure

```
_18_click_clipboard/
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── App.css
    ├── data/
    │   └── items.js
    ├── utils/
    │   └── clipboard.js
    ├── hooks/
    │   └── useCopyToClipboard.js
    └── components/
        ├── CopyField.jsx
        └── CopyButton.jsx
```

11. Full working code

`main.jsx` and `index.css` stay as created by Vite.

`src/utils/clipboard.js`

```
// Modern async API first; fall back to execCommand on http, old browsers, or a denied permission.
export async function copyText(text) {
  if (typeof text !== "string" || text.length === 0) {
    throw new Error("Nothing to copy");
  }
  if (navigator.clipboard && window.isSecureContext) {
    try {
      await navigator.clipboard.writeText(text);
      return;
    } catch {
      // permission denied / document not focused: try the legacy path below
    }
  }
  legacyCopy(text);
}

function legacyCopy(text) {
  const previousFocus = document.activeElement;
  const textarea = document.createElement("textarea");
  textarea.value = text;
```

```
  textarea.setAttribute("readonly", ""); // no virtual keyboard on mobile
  textarea.style.position = "fixed"; // no scroll jump
  textarea.style.opacity = "0";
  document.body.appendChild(textarea);
  textarea.select();
  try {
    if (!document.execCommand("copy")) {
      throw new Error("Copy command was rejected");
    }
  } finally {
    document.body.removeChild(textarea);
    previousFocus?.focus(); // select() stole focus from the button
  }
}

// Fixed bullet count so the mask does not leak the secret's length.
export function maskSecret(value, visible = 4) {
  if (value.length <= visible) return value;
  return "•".repeat(8) + value.slice(-visible);
}
```

`src/hooks/useCopyToClipboard.js`

```
import { useCallback, useEffect, useRef, useState } from "react";
import { copyText } from "../utils/clipboard";

// status: "idle" | "copied" | "error"
export function useCopyToClipboard(resetAfter = 2000) {
  const [status, setStatus] = useState("idle");
  const timerRef = useRef(null);

  useEffect(() => () => clearTimeout(timerRef.current), []);

  const copy = useCallback(
    async (text) => {
      let next = "copied";
      try {
        await copyText(text);
      } catch {
        next = "error";
      }
      // Clear after the await: a second click may have started a timer while we waited.
      clearTimeout(timerRef.current);
      setStatus(next);
      timerRef.current = setTimeout(() => setStatus("idle"), resetAfter);
    },
    [resetAfter]
```

```
  );

  return { status, copy };
}
```

`src/components/CopyButton.jsx`

```
import { FiCheck, FiCopy, FiX } from "react-icons/fi";
import { useCopyToClipboard } from "../hooks/useCopyToClipboard";

const LABELS = { idle: "Copy", copied: "Copied", error: "Failed" };
const ICONS = { idle: FiCopy, copied: FiCheck, error: FiX };

export default function CopyButton({ text, label }) {
  const { status, copy } = useCopyToClipboard();
  const Icon = ICONS[status];

  const announcement =
    status === "copied"
      ? `${label} copied to clipboard`
      : status === "error"
      ? `Could not copy ${label}`
      : "";

  return (
    <>
      <button
        type="button"
        className={`copy-btn copy-btn--${status}`}
        onClick={() => copy(text)}
        aria-label={`Copy ${label}`}
      >
        <Icon aria-hidden="true" />
        <span>{LABELS[status]}</span>
      </button>
      <span className="sr-only" role="status" aria-live="polite">
        {announcement}
      </span>
    </>
  );
}
```

`src/components/CopyField.jsx`

```
import CopyButton from "./CopyButton";
import { maskSecret } from "../utils/clipboard";
```

```
// Displays one value and copies the REAL value, even when the display is masked.
export default function CopyField({ item }) {
  const { label, value, kind } = item;
  const shown = kind === "secret" ? maskSecret(value) : value;

  return (
    <div className="copy-field">
      <span className="copy-field__label">{label}</span>
      <div className="copy-field__row">
        {kind === "code" ? (
          <pre className="copy-field__value">
            <code>{shown}</code>
          </pre>
        ) : (
          <code className="copy-field__value">{shown}</code>
        )}
        <CopyButton text={value} label={label} />
      </div>
    </div>
  );
}
```

`src/data/items.js`

```
// kind decides how the value is DISPLAYED; the copied value is always `value`.
export const COPY_ITEMS = [
  { id: "coupon", label: "Coupon code", value: "DIWALI50", kind: "text" },
  {
    id: "invite",
    label: "Meeting invite link",
    value: "https://meet.google.com/abc-defg-hij",
    kind: "text",
  },
  {
    id: "api-key",
    label: "API key",
    value: "sk_live_51HxQ2eK9vT7mN3pR8wL4aZ1c",
    kind: "secret",
  },
  { id: "otp", label: "One-time password", value: "482913", kind: "text" },
  {
    id: "snippet",
    label: "Setup commands",
    value: "npm install react-icons\nnpm run dev",
    kind: "code",
```

```
  },
];
```

`src/App.jsx`

```
import "./App.css";
import CopyField from "./components/CopyField";
import { COPY_ITEMS } from "./data/items";

export default function App() {
  return (
    <main className="page">
      <h1>Click to Copy</h1>
      <ul className="copy-list">
        {COPY_ITEMS.map((item) => (
          <li key={item.id}>
            <CopyField item={item} />
          </li>
        ))}
      </ul>
    </main>
  );
}
```

`src/App.css`

```
body {
  font-family: system-ui, sans-serif;
  background: #f4f5fb;
  color: #1f2333;
}

.page {
  max-width: 560px;
  margin: 0 auto;
  padding: 40px 16px;
}

h1 {
  font-size: 32px;
  text-align: center;
  margin-bottom: 24px;
}

.copy-list {
  list-style: none;
```

```
  display: grid;
  gap: 12px;
}

.copy-field {
  background: #fff;
  border: 1px solid #e3e5ef;
  border-radius: 12px;
  padding: 14px 16px;
}

.copy-field__label {
  display: block;
  font-size: 13px;
  color: #5b6178;
  margin-bottom: 8px;
}

.copy-field__row {
  display: flex;
  align-items: center;
  gap: 12px;
}

.copy-field__value {
  flex: 1;
  min-width: 0;
  font-family: ui-monospace, Consolas, monospace;
  font-size: 15px;
  background: #f4f5fb;
  border-radius: 8px;
  padding: 8px 10px;
  overflow-x: auto;
  white-space: pre;
}

.copy-btn {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  min-width: 96px;
  justify-content: center;
  padding: 8px 14px;
  border: 1px solid #4f46e5;
  border-radius: 8px;
  background: #fff;
  color: #4f46e5;
```

```
  font-size: 14px;
  cursor: pointer;
  transition: background 0.2s, color 0.2s;
}

.copy-btn:focus-visible {
  outline: 3px solid #a5b4fc;
  outline-offset: 2px;
}

.copy-btn--copied {
  background: #16a34a;
  border-color: #16a34a;
  color: #fff;
}

.copy-btn--error {
  background: #dc2626;
  border-color: #dc2626;
  color: #fff;
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}
```

12. Wrap-up

Edge cases handled

Missing Clipboard API, denied permission, failed fallback, empty value, rapid re-clicks, async click race, unmount during feedback, masked secrets, multi-line text, long values, keyboard and screen reader users, independent feedback per field.

Trade-offs to mention

- **State per button vs one shared `copiedId` in the parent.** Per button keeps renders local and the component drop-in reusable; a shared id would be needed if the product wants "only one Copied at a time".
- **`execCommand` is deprecated** but is still the only fallback for insecure contexts. It is isolated in one function, easy to delete later.

- **Button text change vs toast.** Inline feedback is right next to the user's eyes and needs no global provider; a toast suits bulk actions ("12 rows copied").
- **Fixed-length mask** hides the key length, but users can't eyeball which key it is beyond the last 4 chars.

What to add with more time

- Unit tests: `copyText` with a mocked `navigator.clipboard` (success, reject → fallback, insecure → fallback, fallback fails); hook tests with fake timers (re-click restarts timer, unmount clears it).
- A "reveal" toggle for secrets, and optional auto-clear of the clipboard for sensitive values.
- Rich copy (`navigator.clipboard.write` with `ClipboardItem` for HTML / images).
- Analytics event on copy; a `onCopy` callback prop.
- A shared `<Toast>` for pages with many copy actions.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why do you need a fallback at all? Every modern browser has `navigator.clipboard`.

Browsers have it, but only in a **secure context**. A dev server opened from a phone via `http://192.168.x.x`, intranet tools on plain http, or a cross-origin iframe without `allow="clipboard-write"` get `undefined` or a rejection. The fallback costs ~15 lines and covers those cases.

Q2. What if the requirement is "only one field shows Copied at a time"?

Lift the state up: the parent keeps `copiedId`, and passes `copied={copiedId === item.id}` and `onCopy={() => setCopiedId(item.id)}`. One timer in the parent. Trade-off: every copy re-renders the whole list (fine for tens of items, memoize rows for hundreds).

Q3. Can you read the clipboard too (paste button)?

`navigator.clipboard.readText()` exists but needs an explicit **permission prompt** (and is not supported in every browser, e.g. Firefox limits it). I would not read the clipboard without a clear user action, for privacy.

State design

Q4. Why a `status` string instead of `isCopied` and `isError` booleans?

Two booleans give four combinations, one of them impossible (`copied && error`). An enum makes impossible states unrepresentable and the render a simple lookup: `LABELS[status]`, `ICONS[status]`.

Q5. Why store the timer id in `useRef` and not `useState`?

The id is never shown. `useState` would trigger a useless re-render on every click, and the cleanup closure would see a stale value. A ref is mutable and persistent across renders without rendering.

Q6. Why not store the masked value in state?

It is derived from `value` + `kind`. Storing it creates two sources of truth that can disagree. Compute it during render; it's a cheap string operation.

Challenge: Clipboard API and fallback

Q7. Why check `window.isSecureContext` and not just `navigator.clipboard`?

In insecure contexts `navigator.clipboard` is usually `undefined`, but some environments (old polyfills, test setups) define it anyway and then fail. Checking both is explicit and cheap; the `try/catch` around `writeText` covers the rest.

Q8. Why does `writeText` sometimes fail with "Document is not focused"?

Clipboard writes require **transient user activation** and a focused document. Calling it from a `setTimeout` long after the click, or from DevTools, fails. That's why `copy` runs directly in the click handler.

Q9. Why `readonly`, `position: fixed` and focus restore on the textarea?

`readonly` stops the mobile keyboard from opening; `fixed` stops the page from scrolling to the bottom where the textarea is appended; `select()` moves focus into the textarea, so we put focus back on the button for keyboard users.

Challenge: Timer

Q10. Walk me through the double-click bug if you `clearTimeout` before the `await`.

Click 1: clear (nothing) → await. Click 2: clear (nothing) → await. Both resolve; click 1 sets timer A, click 2 overwrites the ref with timer B. A is never cleared and fires early. Clearing **after** the await means the later resolution always cancels the earlier timer.

Q11. What happens without the unmount cleanup?

The timer fires after the component is gone and calls `setStatus` on an unmounted component. React 18+ ignores it silently, but it's still a leak (closure kept alive) and a bug pattern if the callback does more (analytics, navigation).

Q12. Should the user be able to click while it shows "Copied"?

Yes. Disabling the button would block a legit re-copy and also remove it from the tab order, which moves focus. We just restart the 2 s timer.

Implementation steps

Q13. Why `useCallback` on `copy`?

So consumers can put `copy` in effect dependency arrays or pass it to memoized children without it changing every render. Here it's cheap insurance; it depends only on `resetAfter`.

Q14. Why `type="button"` on the button?

If `CopyButton` is ever placed inside a `<form>`, a button without a type defaults to `submit`, and copying would submit the form.

Q15. Why `aria-live="polite"` and not `assertive`?

"Copied" is a confirmation, not an emergency. `polite` waits for the screen reader to finish its current speech; `assertive` interrupts. The region must be in the DOM **before** its text changes, so it's always rendered, just empty when idle.

Q16. Why use the item `id` as the key and not the index?

State lives inside each `CopyButton`. With index keys, if the list is filtered or sorted, React reuses the component at that index and the "Copied" state jumps to a different field.

Performance and scale

Q17. What if the page has 10,000 copyable rows (a big table of API keys)?

State is per button, so a copy re-renders one button only. The real cost is rendering 10k rows: virtualize the list (`react-window`) so only visible rows mount. The hook and utils don't change.

Q18. What if the value to copy must be fetched first (e.g. "generate invite link")?

Clipboard writes need user activation, and a long `await fetch` can lose it (Safari is strict). Use `navigator.clipboard.write([new ClipboardItem({ "text/plain": fetchPromiseAsBlob })])`, which accepts a promise and keeps the activation; show a `loading` status meanwhile.

Q19. What if the API is slow and the user clicks Copy 5 times?

For the fetch-then-copy case, ignore clicks while `status === "loading"` (or abort the previous request with `AbortController`) so only one request is in flight and only the latest result is copied.

Trade-offs / "why not X?"

Q20. Why not use the `copy-to-clipboard` npm package?

It's the same fallback logic in a dependency. For ~25 lines I keep control over error handling and focus restore and add no bundle weight. In a big codebase that already has it, I would reuse it.

Q21. Why not a toast notification?

A toast needs a global provider and appears far from where the user is looking. Inline button feedback is local, instant and needs no extra infrastructure. Toasts make sense for bulk actions or when the trigger isn't visible anymore.

Q22. Is copying an API key to the clipboard a security risk?

Clipboard contents can be read by other apps and clipboard-history tools. Mitigations: mask on screen, copy only on explicit click, optionally overwrite the clipboard after N seconds, and never log the value. We can't fully control the OS clipboard.